Back End Design
Systems Interaction Design

Invoice Finance DApp

Document Type	Back End Design / System Interactions
Purpose	Describe server-side architecture, actors, components, and operational flows
Version	1.0
Status	Draft for implementation and review

Table of Contents

1. Overview

2. Design Goals

3. Actors and Responsibilities

4. Back End Architecture Overview

5. Core Back End Components

6. Authentication, Session, and Wallet Binding

7. Invoice Lifecycle Interactions

    7.1 Create Invoice

    7.2 Fund Invoice

    7.3 Repay Invoice

8. Transaction Logging and Reporting

9. Security Controls and Trust Boundaries

10. Operational Flows and Interaction Diagrams

11. Data Storage and Persistence

12. API and Integration Endpoints

13. Error Handling and Recovery

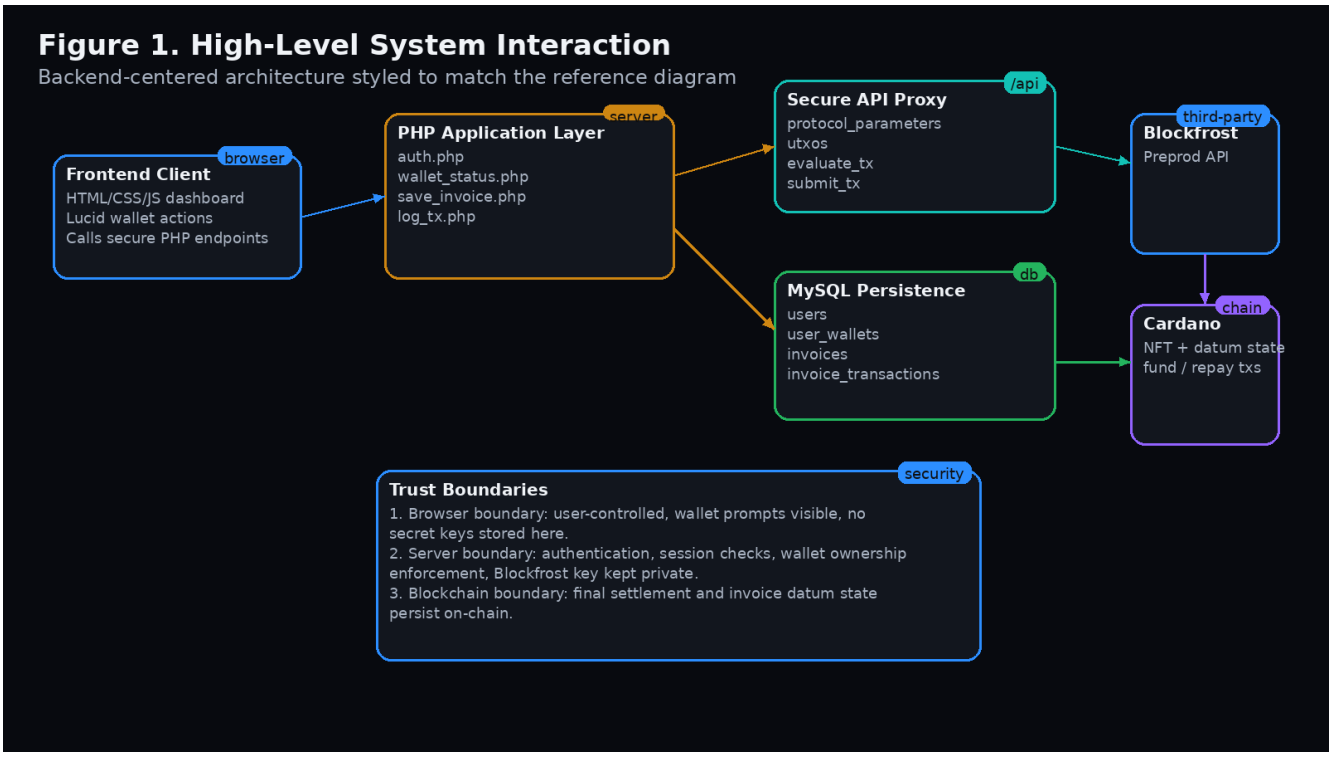
14. Backend Screenshot / Diagram Placeholders

15. Summary

1. Overview

The Invoice Finance DApp back end coordinates account access, wallet binding, invoice persistence, blockchain transaction support, and operational reporting. Its role is to keep sensitive credentials off the client, enforce application rules, and maintain a reliable bridge between the web interface, database records, and Cardano infrastructure.

Figure 1. High-Level System Interaction



2. Design Goals

- Protect sensitive credentials such as the Blockfrost API key by keeping them server-side.
- Enforce user authentication and wallet-to-account binding before blockchain actions are allowed.
- Persist invoice metadata and transaction records in the database for fast rendering and reporting.
- Support clear lifecycle transitions: created, available, funded, repaid, and archived.
- Provide auditable records for administration, debugging, and compliance-style review.

3. Actors and Responsibilities

The template backend interaction document focuses on actor-based system access. For the Invoice Finance DApp, the same pattern is adapted to the following roles:

Actor	Primary Back End Interaction	Typical Permissions
Issuer / SME	Creates invoices, uploads documents, binds wallet, repays	Create own invoices, view own funded obligations, trigger

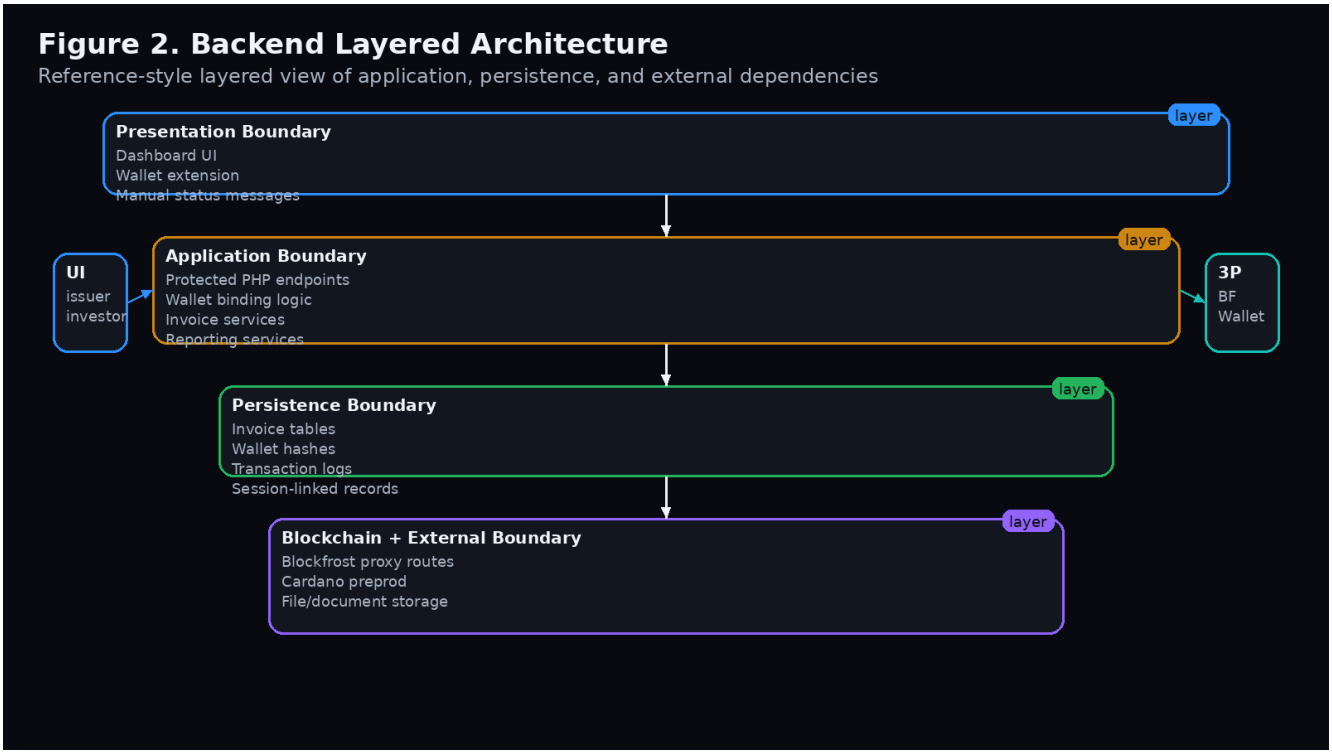
	funded invoices	repayment
Investor / Factor	Connects wallet, browses available invoices, funds selected invoices	View open invoices, fund available invoice, view history
Administrator	Manages users, monitors transactions, reviews operational logs and content	Admin dashboards, user oversight, reporting access
Application Back End	Validates sessions, stores invoices, proxies blockchain requests, logs state changes	Server-side enforcement and orchestration

4. Back End Architecture Overview

The backend uses a layered structure. The browser interacts with PHP endpoints. These endpoints enforce login, wallet ownership, and input validation; store or retrieve records from MySQL; and call external Cardano infrastructure only through controlled server-side proxy endpoints.

- Presentation boundary: browser UI and JavaScript client.
- Application boundary: PHP endpoints for invoices, auth, wallet binding, stats, history, and secure proxy calls.
- Persistence boundary: MySQL tables for users, wallets, invoices, and transaction records.
- Blockchain boundary: Lucid transactions in the browser plus backend-supported provider and monitoring endpoints.
- Third-party boundary: Blockfrost, wallet extensions, and optional email or document services.

Figure 2. Backend Layered Architecture



5. Core Back End Components

Authentication and Session Layer

Handles login state, protected routes, session initialization, CSRF-related controls, and access checks.

Wallet Binding Services

Issues wallet challenges, verifies binding state, and links an approved wallet address hash to the user account.

Invoice Persistence Services

Stores uploaded document paths, invoice references, face value, repayment value, and invoice lifecycle state.

Blockchain Proxy Layer

Provides server-side routes for protocol parameters, UTxO lookup, evaluation, submission, and transaction confirmation checks.

Transaction Logging Layer

Writes create, fund, and repay events into the database for audit and reporting.

Reporting Services

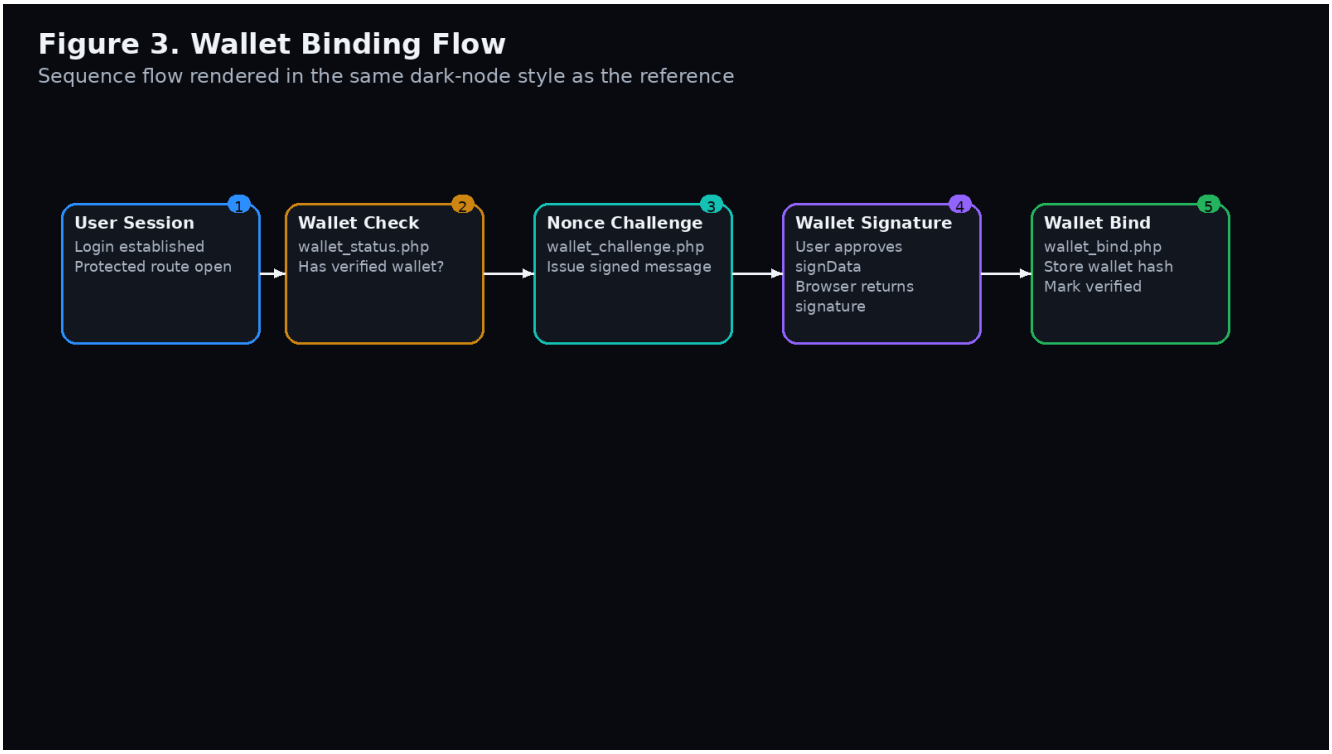
Generates recent activity, wallet-based transaction search, and live dashboard statistics.

6. Authentication, Session, and Wallet Binding

Before any sensitive action is performed, the backend requires a valid authenticated session. Wallet binding then ensures that the connected wallet is the wallet approved for that user account.

- User signs in through standard application authentication.
- Backend checks whether a verified wallet already exists for the account.
- If needed, backend issues a nonce-based challenge for wallet binding.
- Frontend asks the wallet to sign the challenge message.
- Backend stores the verified wallet hash and marks it as approved for that user.
- Subsequent requests compare the connected wallet with the stored approved wallet.

Figure 3. Wallet Binding Flow



7. Invoice Lifecycle Interactions

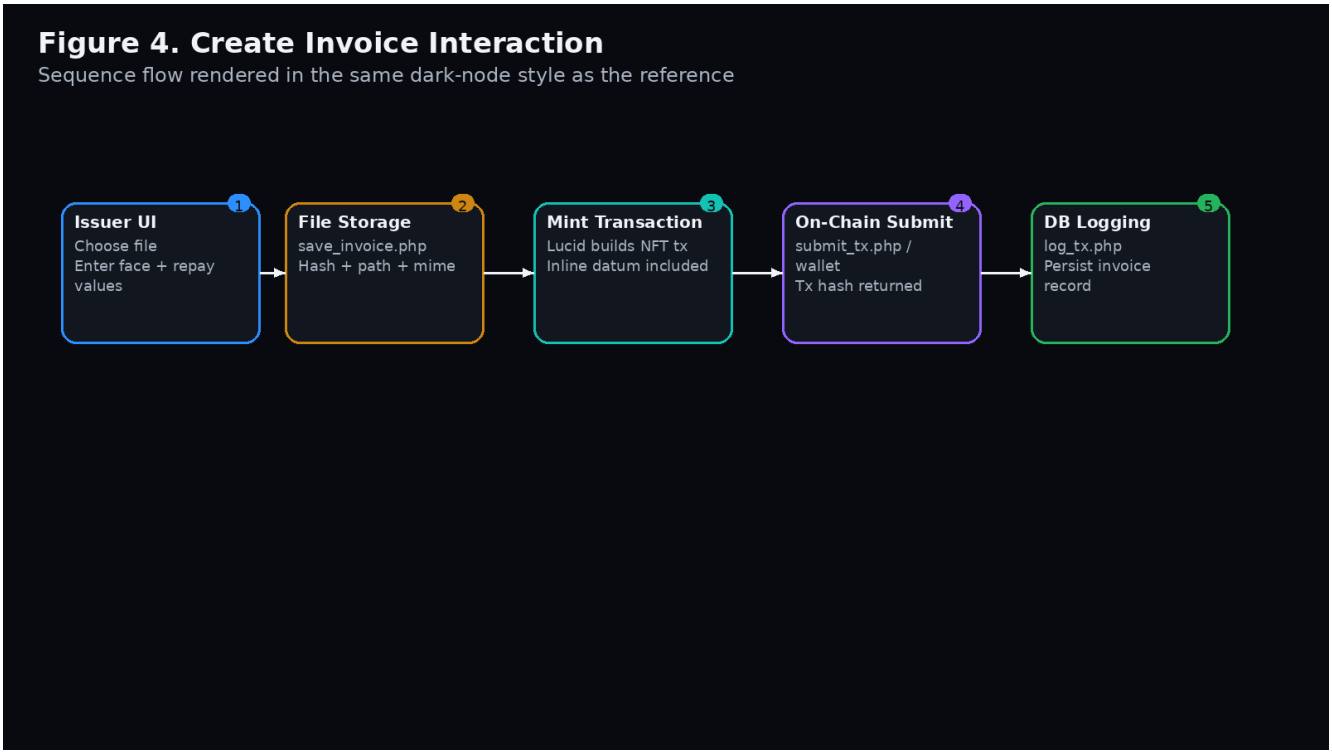
The invoice lifecycle is the central business flow supported by the backend. Each stage produces both on-chain effects and off-chain persistence updates.

7.1 Create Invoice

- Issuer uploads the invoice document through a protected endpoint.
- Backend stores the file and returns a safe document path, mime type, and deterministic hash.
- Frontend builds NFT metadata and mints the invoice on-chain.
- After submission, the backend logs the transaction and persists the invoice reference and document metadata.

- The invoice becomes available for marketplace display through database-backed fetch endpoints.

Figure 4. Create Invoice Interaction



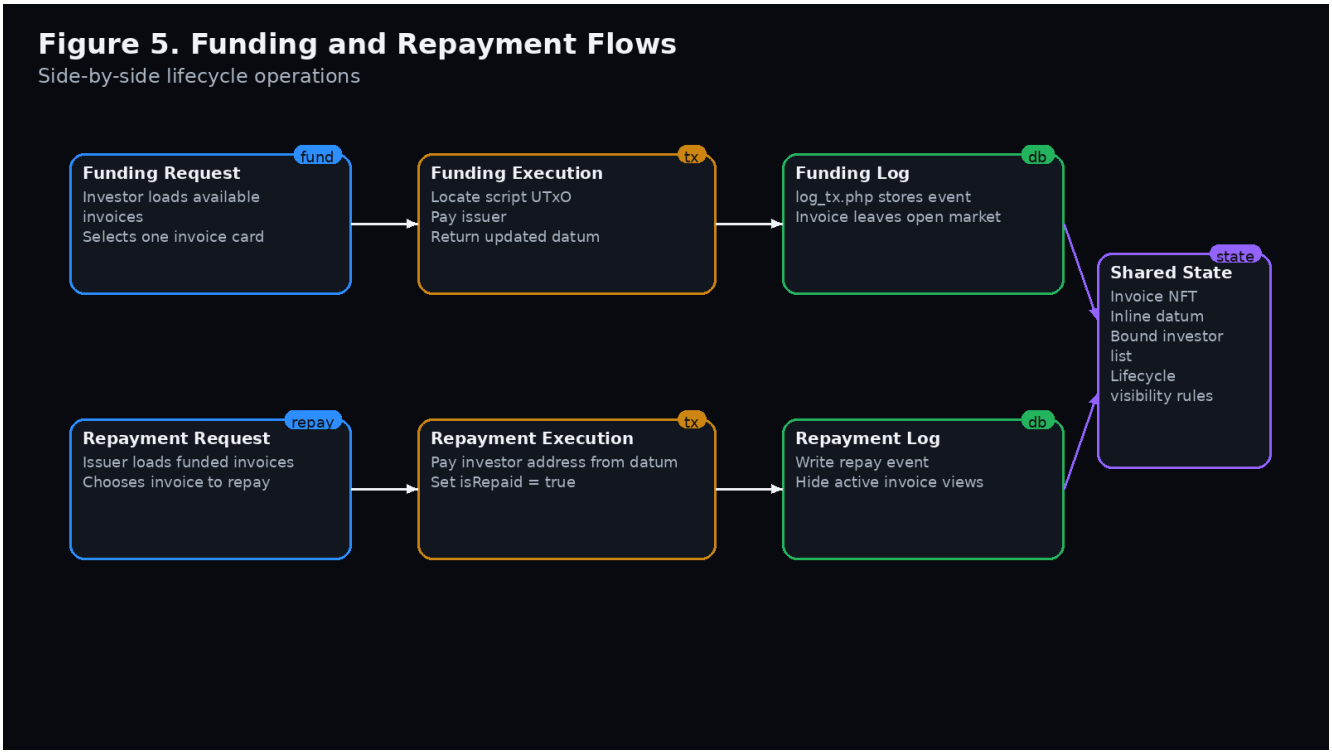
7.2 Fund Invoice

- Investor requests available invoices through database-backed listing endpoints.
- Frontend locates the matching script UTxO and builds the funding transaction.
- After successful submission, the backend records a funding event and updates visibility rules for invoice rendering.
- The funded invoice is removed from the available investor view and appears in the issuer repayment view.

7.3 Repay Invoice

- Issuer fetches funded invoices associated with the bound wallet/account.
- Frontend builds a repayment transaction and distributes payment to investor addresses encoded in datum state.
- Backend records the repayment event in the transaction log.
- Database-backed invoice views stop showing the invoice as active once repayment is complete.

Figure 5. Funding and Repayment Flows



8. Transaction Logging and Reporting

The backend transaction log is essential because blockchain confirmation alone is not sufficient for fast operational views. Application-level records make it possible to populate recent transactions, CSV exports, and search results efficiently.

- Each create, fund, and repay action writes a transaction row with hashes, actor wallet, counterparty wallet, amount, and status.
- Reporting endpoints read from invoice_transactions for dashboard widgets and history pages.
- Wallet hashes are stored alongside raw addresses to support controlled lookups and privacy-aware matching.

9. Security Controls and Trust Boundaries

Control	Risk Addressed	Back End Response
Server-side Blockfrost key storage	API key exposure in browser	Key lives in .env / server environment and is used only by PHP proxy routes
Wallet binding	Wrong user operating another wallet	Backend checks approved wallet hash before allowing session-backed operations
Protected endpoints	Unauthenticated direct access	require_login() gate on sensitive

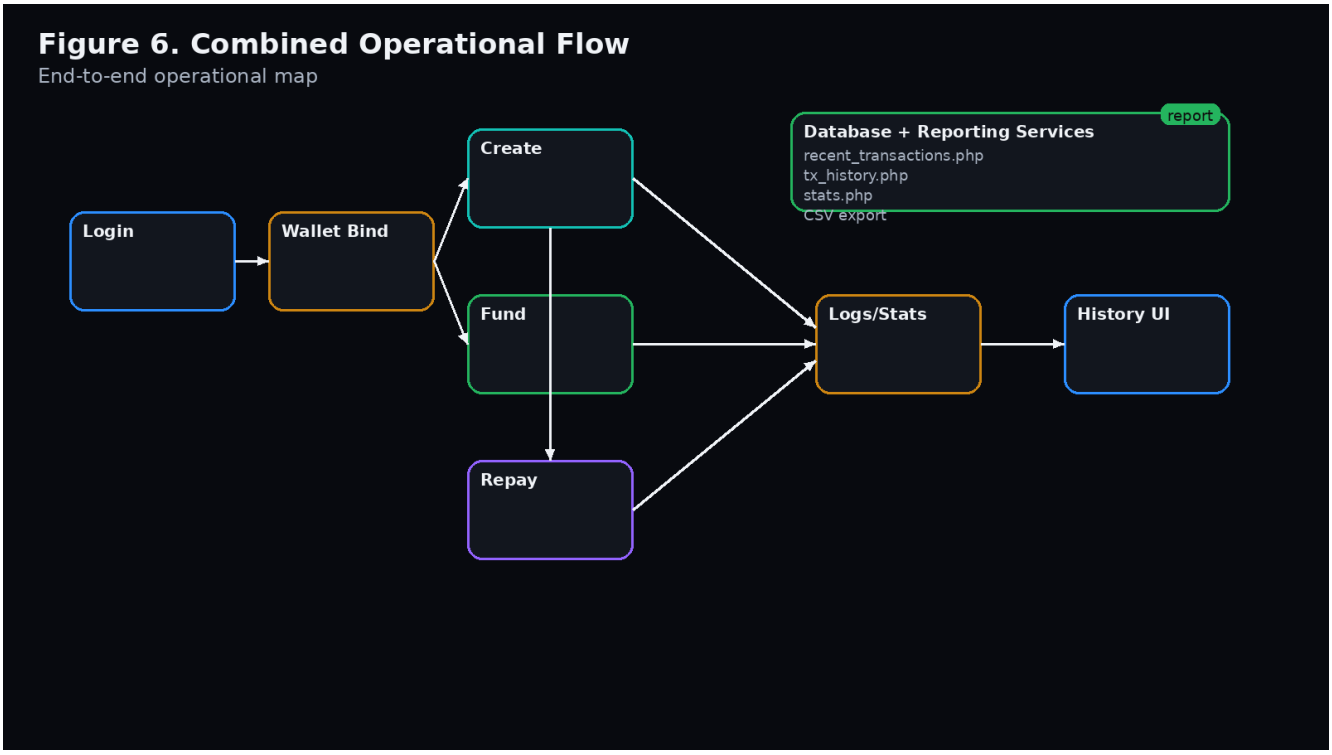
		routes
Input validation	Malformed invoice or wallet data	Trim, validate, reject missing fields, and sanitize file uploads
Transaction logging	Weak audit trail	Persistent log rows for create, fund, repay, and status changes

10. Operational Flows and Interaction Diagrams

The template system interaction document emphasizes how actors enter the system and consume feature areas. For this dApp, the main operational flow groups are authentication, invoice management, investment, repayment, and reporting.

- Authentication flow: login, session creation, wallet status check, optional wallet bind.
- Invoice management flow: upload document, create invoice record, mint and log transaction.
- Investment flow: load open invoices, fund invoice, update list visibility, log event.
- Repayment flow: load issuer-funded invoices, repay investor, mark completed state, log event.
- Reporting flow: fetch recent transactions, address history search, dashboard stats, CSV export.

Figure 6. Combined Operational Flow



11. Data Storage and Persistence

The backend stores durable business data off-chain while keeping only transaction-critical state on-chain. This hybrid model improves performance, simplifies UI rendering, and reduces reliance on repeated third-party fetches.

- On-chain: invoice NFT, datum state, investor binding within datum, repayment condition state.
- Off-chain in MySQL: users, verified wallets, stored document paths, invoice rows, transaction logs, analytics inputs.
- Session state: authenticated user identity, wallet challenge nonce, temporary interaction state.

12. API and Integration Endpoints

Representative backend endpoints for the current system include:

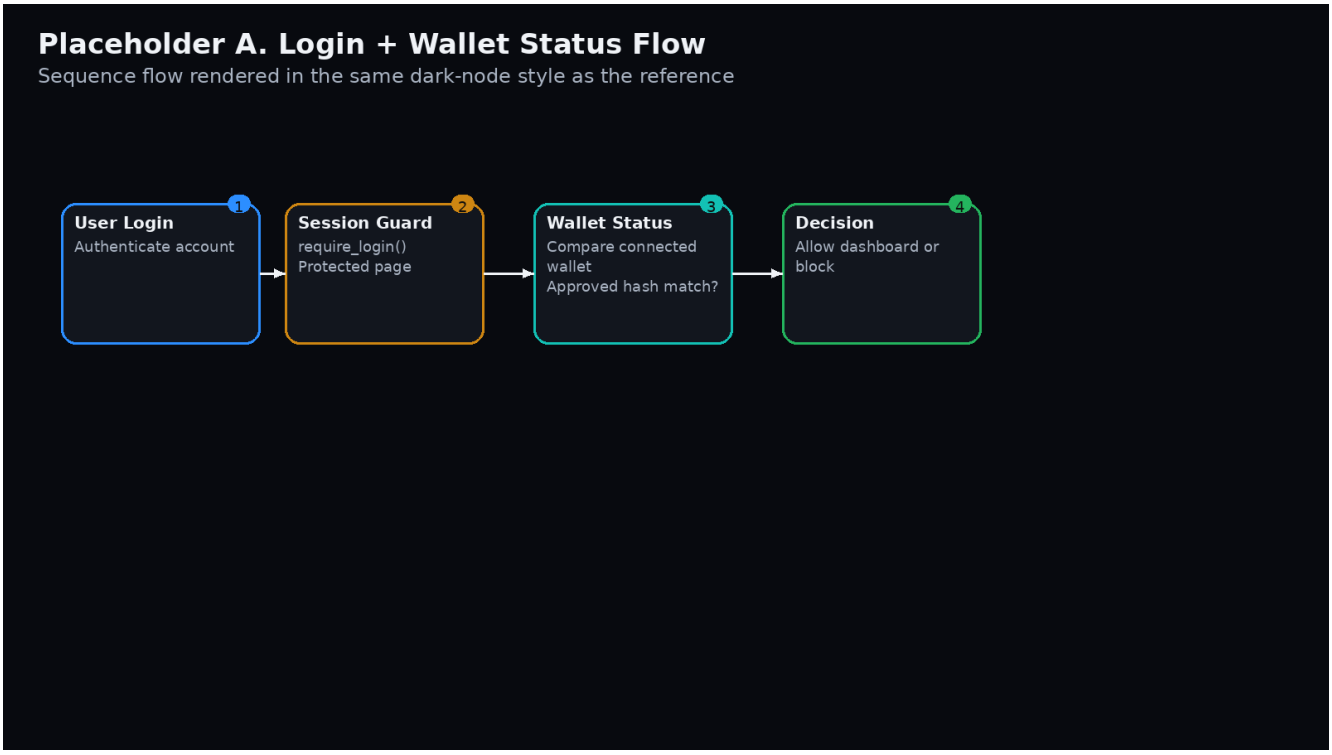
- wallet_status.php – verifies whether the connected wallet is allowed for the session user.
- wallet_challenge.php and wallet_bind.php – manage wallet binding challenge flow.
- save_invoice.php – stores invoice document and returns normalized metadata for minting.
- get_invoices.php and get_my_funded_invoices.php – power database-backed invoice rendering.
- log_tx.php – records business events tied to on-chain transactions.
- recent_transactions.php and tx_history.php – serve operational history and exports.
- Secure /api proxy routes – provide server-side access to protocol parameters, UTxOs, tx evaluation, tx submission, and confirmation checks.

13. Error Handling and Recovery

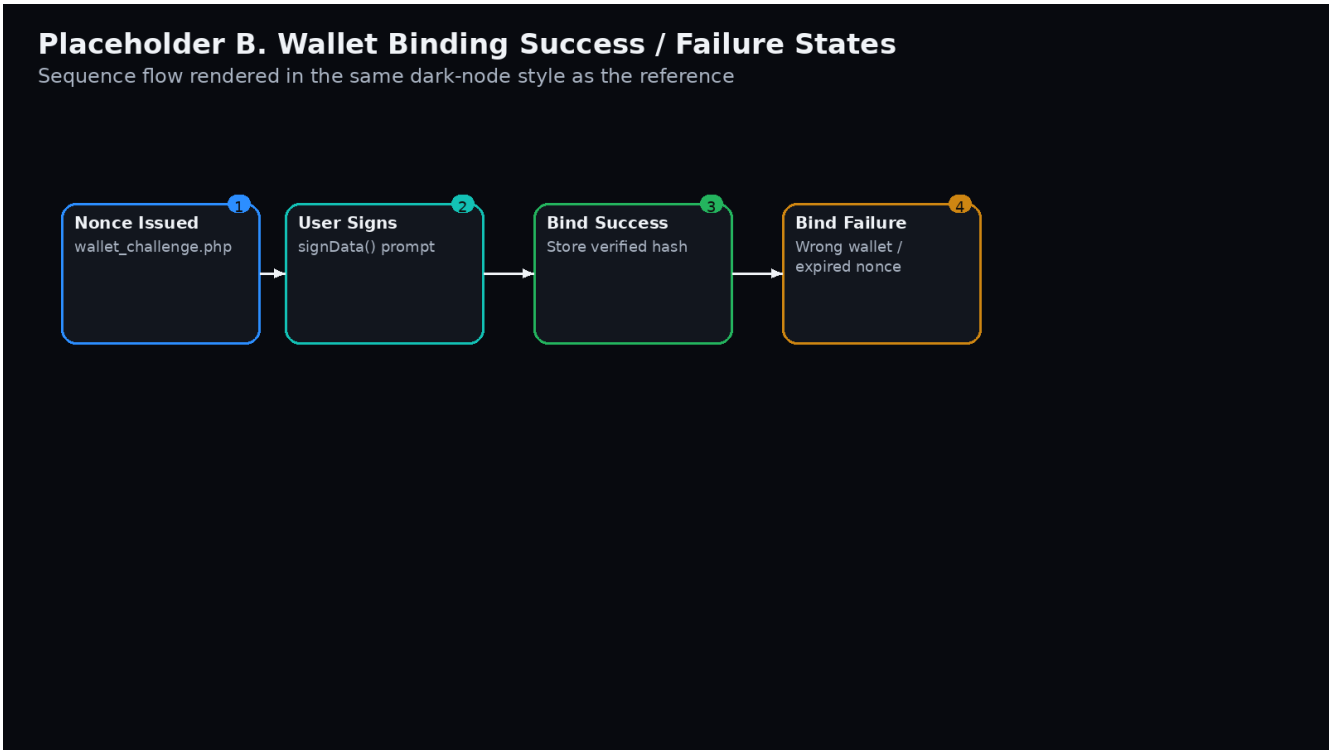
- Authentication failures should redirect the user back to login or show a session-expired message.
- Wallet mismatch should block blockchain actions until the correct wallet is connected.
- Upload failures should not create partial invoice database records.
- Transaction submission failures should be surfaced to the UI and not be silently marked successful in logs.
- History and reporting endpoints should degrade gracefully and show empty-state messages when no records exist.

14. Backend Screenshot / Diagram Placeholders

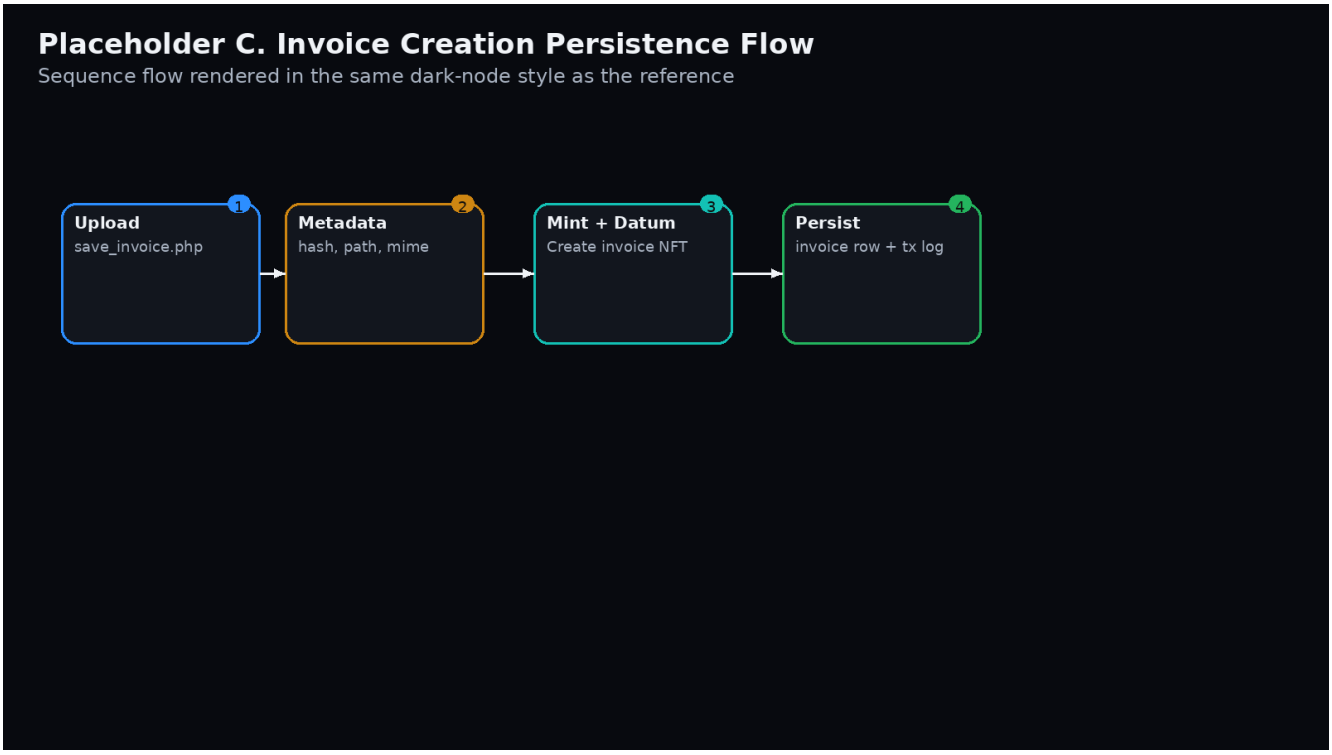
Placeholder A. Login + wallet status flow



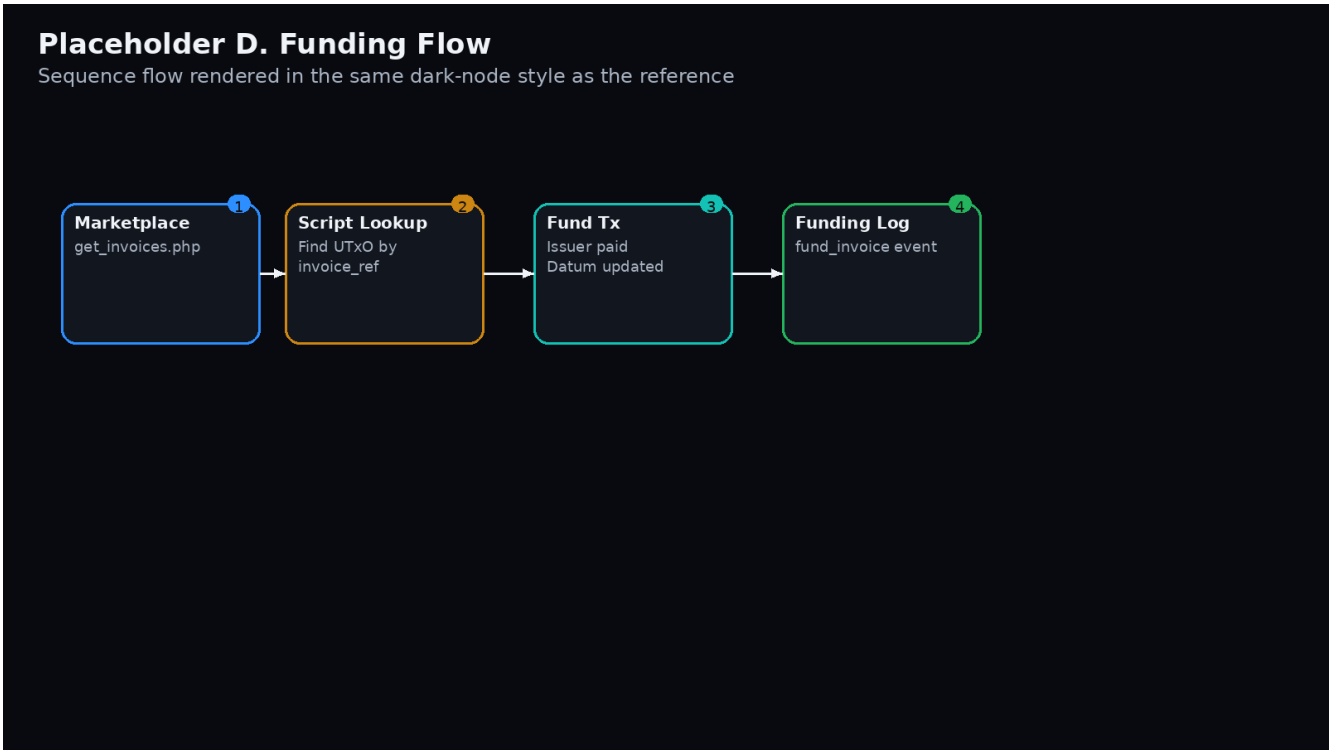
Placeholder B. Wallet binding success / failure states



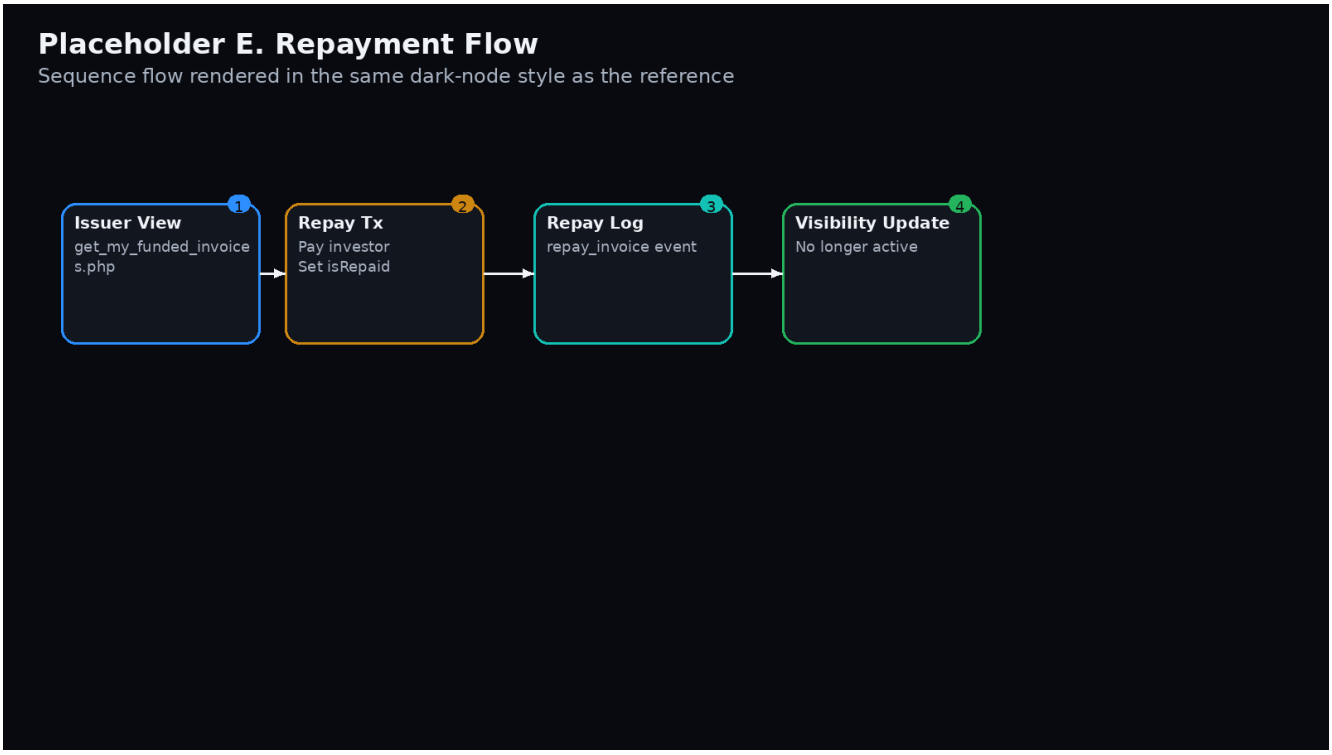
Placeholder C. Invoice creation persistence flow



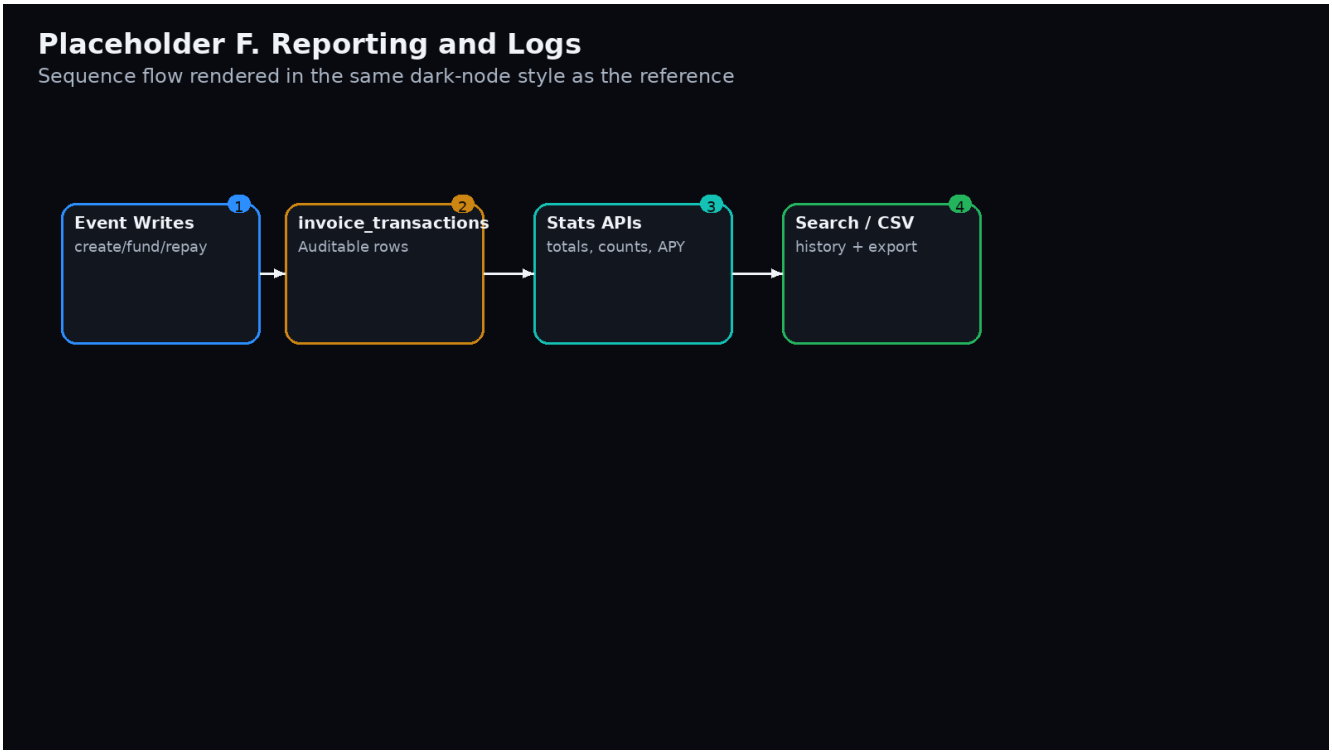
Placeholder D. Funding flow



Placeholder E. Repayment flow



Placeholder F. Reporting and logs



15. Summary

The Invoice Finance DApp backend is designed as a controlled interaction layer between authenticated users, bound wallets, persistent invoice records, and Cardano infrastructure. The system interaction template supplied for reference is therefore extended here into a full backend operating model that supports secure invoice tokenization, funding, repayment, and reporting.